

ARIMA-LSTM Stock Price Prediction

A hybrid time-series forecasting system that combines classical **ARIMA** models with a **deep learning LSTM** network to capture both linear and nonlinear temporal dependencies in stock price data.

Built during the **Association of Computing Activities, IIT Kanpur** (May '25 – July '25)

Overview

Stock price movements exhibit both linear trends/seasonality and nonlinear, complex patterns that a single model often struggles to capture. This project addresses that gap by:

- Using **ARIMA** (and its simpler variants AR, MA, ARMA) to model the linear component of the series
 - Feeding the residuals / raw series into an **LSTM** network to learn nonlinear temporal dependencies
 - Combining both signals into a hybrid **ARIMA-LSTM** forecast
-

Objective

To accurately forecast stock prices by leveraging the complementary strengths of classical time-series modeling and modern deep learning.

Approach

1. Data Collection & Exploration

- Collected historical stock price data using **yfinance**
- Performed exploratory data analysis and result visualization using **Matplotlib** and **Seaborn**

2. Modeling

- Designed and compared classical time-series models: **AR, MA, ARMA, and ARIMA**
- Developed a hybrid **ARIMA-LSTM** architecture using **statsmodels** (for ARIMA) and **TensorFlow** (for LSTM) to jointly capture linear and nonlinear temporal dependencies

3. Training & Tuning

- Split data **chronologically** into train/test sets to preserve temporal order and avoid look-ahead bias
- Tuned LSTM hyperparameters — epochs, batch size, sequence length — via **grid search**, optimizing for minimum validation RMSE

Results

Metric	Value
RMSE (total data)	2.25
R ² (best case)	up to 0.91
Accuracy improvement (hybrid vs. baseline)	+25%

The hybrid ARIMA-LSTM model outperformed the standalone classical models (AR, MA, ARMA, ARIMA), demonstrating that combining linear and nonlinear approaches yields more accurate stock price forecasts.

Tech Stack

- **Python**
 - **yfinance** — historical data collection
 - **statsmodels** — AR / MA / ARMA / ARIMA modeling
 - **TensorFlow / Keras** — LSTM network
 - **Matplotlib, Seaborn** — data exploration & visualization
 - **scikit-learn** — evaluation metrics (RMSE, R²)
-

Project Structure

```
|— data/           # Raw and processed stock price data
|
|— notebooks/      # EDA and experimentation notebooks
|
|— src/
|   |— data_collection.py  # yfinance data pull
|   |— arima_models.py    # AR / MA / ARMA / ARIMA implementations
|   |— lstm_model.py      # LSTM architecture & training
|   |— hybrid_model.py    # ARIMA-LSTM combination logic
|   |— evaluate.py        # RMSE / R2 evaluation
|
|— results/        # Saved plots, metrics, model outputs
|
|— README.md
```

How to Run

pip install yfinance statsmodels tensorflow scikit-learn matplotlib seaborn

```
python src/data_collection.py    # fetch historical stock data
python src/arima_models.py       # fit AR/MA/ARMA/ARIMA baselines
python src/lstm_model.py         # train LSTM
python src/hybrid_model.py       # train combined ARIMA-LSTM
python src/evaluate.py           # compute RMSE / R2 on test set
```

Key Takeaways

- Chronological train/test splitting is essential for realistic time-series evaluation

- Hybrid statistical + deep learning models can meaningfully outperform either approach alone
- Careful hyperparameter tuning (via grid search) on sequence length and batch size had a significant impact on validation RMSE